Machine Discovery of Mathematical Conjectures and Structures

DSA5210 Session 09 —AI for Mathematics

10 June 2026

Relations among numbers

PSLQ: finding a relation among numbers

PSLQ detects **integer relations**. Given reals $x_1, \ldots, x_n$ to high precision, it returns integers $a_1, \ldots, a_n$ (not all zero) with

$$a_1 x_1 + a_2 x_2 + \cdots + a_n x_n = 0,$$

or certifies that none exists below a height bound.

- Ferguson–Forcade (1979); the stable PSLQ is Ferguson–Bailey–Arno (1999); related to LLL (Lenstra–Lenstra–Lovász) lattice reduction.
- Feed it a constant's value plus candidate terms (π, logs, ζ values, powers); it returns an exact-looking identity.
- Named one of the “top ten algorithms of the century” (2000).

The identity is a candidate: true to the computed precision, still to be proved.

An integer relation is a short vector in a lattice

Embed the numbers: take $\mathbf{b}_i = (\mathbf{e}_i,\, Nx_i)$, where $\mathbf{e}_i = (0, \ldots, 1, \ldots, 0)$ is the i-th unit vector and N is large. Each $\mathbf{b}_i$ is an $(n+1)$-vector, so

$$\sum_i a_i \mathbf{b}_i = \big(\underbrace{a_1, \ldots, a_n}_{\text{from the } \mathbf{e}_i},\ \underbrace{N \textstyle\sum_i a_i x_i}_{\text{last coordinate}}\big).$$

If $\sum_i a_i x_i \approx 0$ with small a_i, this lattice vector is **short**.

So **an integer relation = a short vector in this lattice**, and the goal is to find the shortest one. That is what lattice reduction (LLL) does.

LLL: reduce the basis

Input a lattice basis $\mathbf{b}_1, \ldots, \mathbf{b}_n$.

Output a basis of the same lattice with a short first vector, $\|\mathbf{b}_1\| \le 2^{(n-1)/2}\lambda_1$ ($\lambda_1 =$ true shortest length); $\mathbf{b}_1$ is the candidate relation.

Minimizes the potential $D = \prod_k \prod_{i \le k} \|\mathbf{b}_i^*\|^2$ (product of the partial Gram determinants).

($\mathbf{b}_i^*$: Gram–Schmidt vectors; $\mu_{ij} = \langle \mathbf{b}_i, \mathbf{b}_j^* \rangle / \langle \mathbf{b}_j^*, \mathbf{b}_j^* \rangle$.)

A descent on D, alternating two moves until neither applies:

- **Size-reduce**: $\mathbf{b}_i \leftarrow \mathbf{b}_i - \lfloor \mu_{ij} \rceil \mathbf{b}_j$ until $|\mu_{ij}| \le \frac{1}{2}$ — near-orthogonal to earlier vectors; D fixed.
- **Lovász swap**: if $\|\mathbf{b}_i^*\|^2 < (\delta - \mu_{i,i-1}^2)\|\mathbf{b}_{i-1}^*\|^2$ ($\delta = \frac{3}{4}$), swap $\mathbf{b}_{i-1} \leftrightarrow \mathbf{b}_i$ — the shorter vector moves up; D drops by $\le \delta$, so it stops.

PSLQ: input, output, objective

Input reals $x_1, \ldots, x_n$ to high precision.

Output a nonzero integer vector $\mathbf{a}$ with $\sum_i a_i x_i = 0$; or a bound M with every relation $\|\mathbf{a}\| \geq M$ (none below height M).

Objective the smallest integer relation, $\min \|\mathbf{a}\|$ over $0 \neq \mathbf{a} \in \mathbb{Z}^n$ with $\sum_i a_i x_i = 0$ — the same shortest vector LLL seeks.

The difference from LLL is in *how*: PSLQ reduces the real numbers x_i directly, kept to high precision, instead of building LLL's integer lattice $\mathbf{b}_i = (\mathbf{e}_i, N x_i)$. So there is no large multiplier N to pick or inflate, and the arithmetic stays numerically stable.

PSLQ: the matrix $H_{\mathbf{x}}$

Every integer relation lies in the **relation space** $\mathbf{x}^{\perp} = \{\mathbf{v} \in \mathbb{R}^n : \mathbf{v} \cdot \mathbf{x} = 0\}$. Normalize $|\mathbf{x}| = 1$ and set $s_k = \sqrt{x_k^2 + \cdots + x_n^2}$ (so $s_1 = 1$). These define the $n \times (n-1)$ matrix

$$(H_{\mathbf{x}})_{jj} = \frac{s_{j+1}}{s_j}, \quad (H_{\mathbf{x}})_{ij} = -\frac{x_i x_j}{s_j s_{j+1}} \ (i > j), \quad (H_{\mathbf{x}})_{ij} = 0 \ (i < j),$$

whose $n-1$ columns are orthonormal and orthogonal to $\mathbf{x}$ — a basis of $\mathbf{x}^{\perp}$.

Certificate (Ferguson–Bailey–Arno): for any invertible integer matrix A and orthogonal Q with $AH_{\mathbf{x}}Q$ lower-trapezoidal (zero above the diagonal), every relation $\mathbf{m}$ satisfies $|\mathbf{m}| \geq 1/\max_j |(AH_{\mathbf{x}}Q)_{jj}|$.

PSLQ: the loop

Keep an integer matrix A (with its inverse A^{-1}) and an orthogonal Q; write $H = AH_{\mathbf{x}}Q$. Fix a parameter $\gamma > 2/\sqrt{3}$, then repeat:

- **Reduce** subtract rounded integer multiples of earlier rows ($A \leftarrow DA$, D integer) until $|H_{ij}| \le \frac{1}{2}|H_{jj}|$ for $i > j$.
- **Exchange** pick r maximizing $\gamma^r|H_{rr}|$; swap rows $r, r+1$ of A.
- **Corner** the swap breaks the lower-trapezoidal form; a plane (Givens) rotation $Q \leftarrow QP$ on columns $r, r+1$ restores it (the "LQ").

Stop when $\mathbf{x}A^{-1}$ has a zero entry: that column of A^{-1} is the relation $\mathbf{m}$. Until then $1/\max_j |H_{jj}|$ lower-bounds every relation — the "no relation below height M" certificate.

A worked discovery: the BBP formula for π

The Bailey–Borwein–Plouffe (BBP) formula:

$$\pi = \sum_{k=0}^{\infty} \frac{1}{16^k}\left(\frac{4}{8k+1} - \frac{2}{8k+4} - \frac{1}{8k+5} - \frac{1}{8k+6}\right)$$

- Found numerically in 1995 (Plouffe) by an integer-relation search; published 1997; then **proved**.
- Gives the n-th hexadecimal digit of π — and the $4n$-th binary digit — without the earlier digits, in little memory.
- Base 16, not base 10: cheap decimal-digit extraction is still open.

Discovered from the numbers, proved afterward.

PSLQ: rediscovering the BBP formula

Feed PSLQ the eight building blocks $S_j = \sum_{k\geq 0} 1/(16^k(8k+j))$, with no prior knowledge of the coefficients:

```
import mpmath as mp;  mp.mp.dps = 100
S = [mp.nsum(lambda k, j=j: 1/(mp.mpf(16)**k*(8*k+j)), [0, mp.inf])
     for j in range(1, 9)]              # S1 .. S8
rel = mp.pslq([mp.pi] + S, maxcoeff=10**6)
```

Output:

```
rel = [1, -4, 0, 0, 2, 1, 1, 0, 0]   ==  published BBP
  ->  pi = 4*S1 - 2*S4 - S5 - S6        (rel. error vs pi: 9e-102)
spigot: pi hex digits at offset 999 = 349F1C   (matches mpmath)
PSLQ finds nothing below 16 digits, then locks on (height 4)
```

The exact formula comes back from the numbers alone; the spigot step then extracts the n-th hexadecimal digit of π directly.

From computed numbers to a closed form: OEIS, LIReC

- **OEIS**, the On-Line Encyclopedia of Integer Sequences (oeis.org; Sloane; over $390{,}000$ sequences) — type the first terms of a computed sequence; it returns matches with conjectured formulas, generating functions, recurrences, and references.
- **LIReC**, the Library of Integer RElations and Constants (github.com/RamanujanMachine/LIReC) — a database of constants and a hypergraph of PSLQ-found relations; a 2024 sweep reported 75 previously unknown connections, including new π–e relations.

LIReC: recovering an integer relation

Three constants to 190 digits — $\zeta(3)$, $\zeta(5)$, and $x = 2 + 3\zeta(3) - 7\zeta(5)$ built from them — handed to LIReC's PSLQ core:

```
from LIReC.lib.pslq_utils import PreciseConstant as C, check_consts
import mpmath as mp;  mp.mp.dps = 200
z3 = C(mp.zeta(3), 190, 'zeta3')
z5 = C(mp.zeta(5), 190, 'zeta5')
x  = C(2 + 3*mp.zeta(3) - 7*mp.zeta(5), 190, 'x')
check_consts([z3, z5, x], degree=1)
```

Output (run on the cluster, mu012):

```
[ x - 3*zeta3 + 7*zeta5 - 2 = 0   (188) ]
```

The integer relation comes back exactly, valid to 188 digits. Put an unknown constant in place of x and the same call is a discovery.

Formulas from data

Symbolic regression: a formula from data

Input numeric pairs (x_i, y_i) and an operator set $\{+, -, \times, \div, \exp, \log, \sqrt{\ }\}$.

Output a *Pareto front* of closed-form formulas f — for each complexity, the best-fitting one.

Minimizes two quantities jointly — the data misfit $\sum_i \big(f(x_i) - y_i\big)^2$ and the expression complexity (number of operators/nodes in the formula). They trade off, so the answer is a front, not one formula.

Algorithm: genetic programming over expression trees — keep a population of candidate formulas, repeatedly mutate and recombine them, and select by a fitness that rewards low error and small size.

- **PySR** (Cranmer, 2023) is this implementation; AI Feynman and SINDy (sparse identification of nonlinear dynamics) are relatives.
- It returns the functional form — no proof, no error bound; a fitted constant is pinned afterward by an integer-relation step.

Partition asymptotics

Let $p(n)$ count the partitions of n — the ways to write n as a sum of positive integers, order ignored. Its generating function is

$$\sum_n p(n)x^n = \prod_{k\geq 1}(1-x^k)^{-1},$$

which lets $p(n)$ be computed exactly in integer arithmetic, with no rounding, up to $n = 20{,}000$.

The Hardy–Ramanujan formula (1918) is

$$p(n) \sim \frac{1}{4n\sqrt{3}}\,\exp\left(\pi\sqrt{\tfrac{2n}{3}}\right).$$

Discovery task: compute $p(n)$, take logarithms, run symbolic regression on $(n, \log p(n))$.

Euler's pentagonal number theorem

The product $\prod_{k\geq 1}(1-x^k)$ — the reciprocal of the partition generating function — expands to an almost-empty signed sum:

$$\prod_{k\geq 1}(1-x^k) = \sum_{j=-\infty}^{\infty} (-1)^j \, x^{j(3j-1)/2} = 1 - x - x^2 + x^5 + x^7 - x^{12} - x^{15} + \cdots$$

Non-zero terms sit only at the generalized pentagonal numbers $\frac{k(3k\pm1)}{2} = 1, 2, 5, 7, 12, 15, \ldots$, with coefficient $(-1)^k$ on the k-th pair.

The coefficient of x^n counts the partitions of n into distinct parts with sign $(-1)^{\#\text{parts}}$; even and odd cancel in pairs (Franklin's bijection, 1881), leaving ± 1 only at a pentagonal n.

Matching x^n coefficients in $\left(\sum_n p(n)x^n\right)\prod_k(1-x^k) = 1$ gives the exact recurrence

$$p(n) = p(n-1) + p(n-2) - p(n-5) - p(n-7) + p(n-12) + \cdots\,,$$

an integer sum over the $O(\sqrt{n})$ pentagonal numbers below n.

Symbolic regression recovers the Hardy–Ramanujan form

Run PySR on $(n, \log p(n))$ with no form supplied — four random seeds in parallel on the CPU cluster, minutes each. Three of the four independently recover (complexity 11, loss $\sim 10^{-10}$):

$$\log p(n) \approx 2.565099\,\sqrt{n - 0.349} - \log n - 1.9355.$$

- $\sqrt{n}$ coefficient 2.565099 vs $\pi\sqrt{2/3} = 2.5650997$; constant -1.9355 vs $-\log(4\sqrt{3}) = -1.93560$ — 5–7 significant figures.
- The -0.349 inside the root is a sub-leading correction, beyond the crude 1918 leading term.
- The proof of the formula is separate from the fit: the circle method, the modular transformation of the generating function, Rademacher's exact series (1937).

The improved asymptotic: Rademacher's leading term

The 1918 formula is only the crude leading term. The first term of Rademacher's exact series (1937) is

$$p(n) \sim \frac{\sqrt{12}}{24n-1}\left(1-\frac{1}{\mu}\right)e^{\mu}, \qquad \mu = \frac{\pi}{6}\sqrt{24n-1},$$

with relative error $\sim 10^{-14}$ by $n = 2000$, where the crude term is still 1% off.

The fitted -0.349 shift is built from two refinements inside this term:

- $24n-1 = 24\big(n-\frac{1}{24}\big)$, so $\mu = \pi\sqrt{\frac{2}{3}}\sqrt{n-\frac{1}{24}}$: the root is shifted by $\frac{1}{24}$.
- the factor $1-\frac{1}{\mu}$ contributes a further $O(1/\sqrt{n})$ term.

Folded into a single $\sqrt{}$-shift, the two give $\frac{1}{24}+\frac{3}{\pi^2} = 0.346$, against the fitted 0.349.

The Pareto front for $\log p(n)$

Symbolic regression returns the best formula at each complexity, not a single answer. For this run:

complexity	best formula	loss
4	$2.460\,\sqrt{n}$	7.5
9	$2.5652\,\sqrt{n} - \log n - 1.952$	1.0×10^{-5}
11	$2.565099\,\sqrt{n - 0.349} - \log n - 1.9355$	2.7×10^{-10}

Complexity 11 is the knee: past it, extra terms cut the loss by only $\sim 10^{-10}$. The complexity-4 row, $2.460\,\sqrt{n}$, is the leading term by itself — the search reaches the full form only after it adds the $-\log n$ term.

Recovering the constant by least squares

With the form assumed, the coefficients follow from a linear least-squares fit of $a\sqrt{n} + b\log n + c$ to $(n, \log p(n))$, $50 \le n \le 20{,}000$:

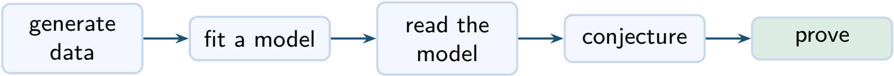
fit	a	ground truth	error
$a\sqrt{n} + b\log n + c$	2.56491	$\pi\sqrt{2/3} = 2.56510$	rel. 7×10^{-5}
$a\sqrt{n}$ only	2.460	same	4%

The full form pins $\pi\sqrt{2/3}$ to five significant figures; the leading term alone is 4% off — the same 2.460 the symbolic-regression front shows at complexity 4. The $-\log n$ term, from the $1/n$ prefactor in the Hardy–Ramanujan formula, is what fixes the constant.

Rules from labelled data

The pipeline: fit a model, then read it

Supervised learning can surface a hidden rule. The pipeline is the one Davies and collaborators used for knot theory and Kazhdan–Lusztig polynomials (Nature 2021).



- Exact labels from a computer algebra system — no measurement noise.
- Read the model through feature importance or saliency — which inputs move the output.
- The machine stops at the conjecture; a human proves it.

Decision tree: a rule from labelled data

Input labelled triples — each (a, b, c) tagged appears (1) or absent (0), presented as raw spins or as difference features.

Output a tree of one-variable threshold tests ("is $a + b - c < 0$?"), a label at each leaf; each root-to-leaf path is an explicit rule.

Minimizes at each node, the weighted Gini impurity of the two children — $\mathrm{Gini} = 1 - p^2 - (1-p)^2 = 2p(1-p)$ for a node with appears-fraction p, and 0 at a pure leaf. Equivalently, it maximizes the impurity drop.

Algorithm: greedily pick the single feature and threshold that minimizes that weighted child impurity; recurse on each branch until the leaves are pure (or a depth cap). The model is white-box — the rule reads off the tree.

Two other readouts of a fitted model: inductive logic programming returns a logical clause; a black-box classifier is read by saliency, the gradient of the output with respect to each input.

Clebsch–Gordan selection rules — the toy

For $SU(2)$, the tensor product of spin-a and spin-b decomposes as

$$V_a \otimes V_b = \bigoplus_{c=|a-b|}^{a+b} V_c,$$

in integer steps; each multiplicity is 0 or 1. It is 1 exactly when

$$|a-b| \leq c \leq a+b \quad \text{and} \quad a+b+c \in \mathbb{Z}$$

— a triangle inequality plus an integrality condition.

Rediscover it: label all triples (a, b, c) with $a, b \leq 10$ from an independent computation of the multiplicity (about $11{,}000$ triples, 30% positive, so "always absent" already scores 0.70), and train a decision tree.

Raw spins vs differences

The outcome turns entirely on how each triple is presented to the tree.

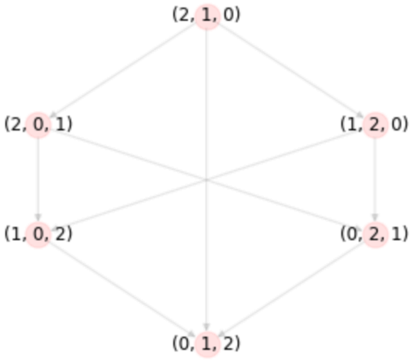
- **Raw (a, b, c):** a depth-12 tree scores 0.58, below the 0.70 baseline — the integrality condition is not a single-coordinate threshold.
- **Differences** $(\,a{+}b{-}c,\ c{-}|a{-}b|,\ [\,a{+}b{+}c \in \mathbb{Z}\,]\,)$: each part of the rule is now one threshold; a depth-4 tree is exact, and trained on $a, b \leq 6$ it still scores 1.00 on larger spins.

The depth-4 tree reads back as the rule:

$$\text{appears} \iff a + b + c \in \mathbb{Z} \ \wedge \ c \geq |a - b| \ \wedge \ c \leq a + b.$$

The symmetric group and the Bruhat graph

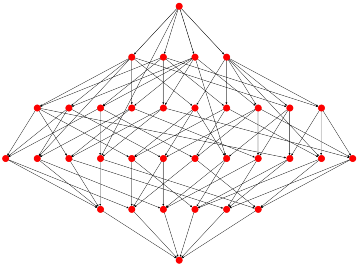
- The symmetric group S_N is all permutations of $\{0, 1, \ldots, N-1\}$, written in string notation: 2031 is the permutation $0 \mapsto 2,\ 1 \mapsto 0,\ 2 \mapsto 3,\ 3 \mapsto 1.$
- Bruhat graph: one node per permutation; an edge joins two permutations that differ by a transposition — a swap of two of the values.
- The length $\ell(x) = \#\{\, i < j : x(i) > x(j) \,\}$ counts inversions. It gives each node a height and orients every edge downward, from longer to shorter.



The ordered Bruhat graph of S_3: top is the reverse permutation 210, bottom is the identity 012.

Bruhat order and the interval $[x, y]$

- Bruhat order: $x \leq y$ when there is a downward path from y to x in the Bruhat graph. The identity is the least element, the reverse permutation w_0 the greatest.
- The interval $[x, y]$ is the set of permutations z with $x \leq z \leq y$, together with the edges among them — a directed graph in its own right.
- Erasing the vertex names leaves the *unlabelled* interval: the shape of that graph alone.



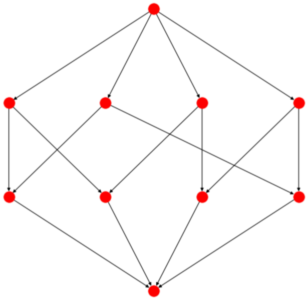
An interval $[03214, 34201]$ inside S_5.

Kazhdan–Lusztig polynomials

- Kazhdan and Lusztig (1979) attached to each pair $x \leq y$ a polynomial $P_{x,y}(q)$ with integer coefficients.
- The definition is inductive on length — computing $P_{x,y}$ may call on $P_{u,v}$ for many pairs below. It is awkward by hand, but a computer generates billions.
- Two facts already tie the polynomial to the graph alone:
  - $P_{x,y} \neq 0$ exactly when $x \leq y$;
  - $P_{x,y} = 1$ exactly when the unoriented interval $[x, y]$ is regular (every vertex has the same degree).
- Why they matter: $P_{x,y}$ records the local intersection cohomology of Schubert varieties (Kazhdan–Lusztig 1980) and composition multiplicities of Verma modules in representation theory.
- Small values: $1, \quad 1+q, \quad 1+q^2, \quad 1+3q+q^2$.

The combinatorial invariance conjecture

- Conjecture (Lusztig and Dyer, independently, 1980s): $P_{x,y}$ depends only on the isomorphism type of the unlabelled interval $[x, y]$ — not on which permutations x and y are.
- Example in S_4: the intervals $[0213, 2301]$ and $[1032, 3120]$ are different, yet both are the same directed graph, the "4-crown", and both have $P_{x,y} = 1 + q$.
- Known when x is the identity; open in general.



The "4-crown". It occurs as two different intervals in S_4, both with KL polynomial $1 + q$.

Graph neural networks

- A graph neural network computes on a graph directly. Each node holds a vector of numbers, started from its features — here its in-degree and out-degree.
- **Message passing:** in each round every node replaces its vector by combining its own with an aggregate (sum or mean) of its neighbours' vectors, through a small learned function. Four rounds were used, so each node ends up summarising its four-hop neighbourhood.
- **Readout:** the node vectors are pooled into one graph-level output — here the predicted coefficients of $P_{x,y}$.
- One set of weights is shared across all nodes and all graphs, so a single network handles intervals of any size and depends only on the connectivity, not on the node names.

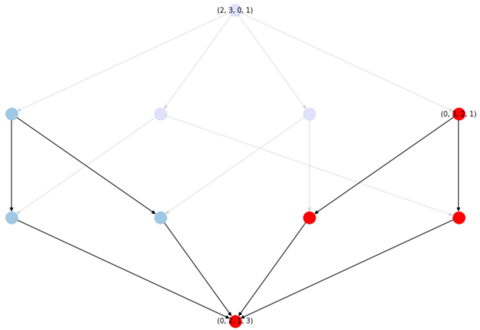
Learning the polynomial from the graph

- **Data:** every interval in the symmetric groups up to S_9 — about 2.9 million in S_9, reduced to $24{,}322$ distinct interval graphs; split $80/20$ into train and test.
- **Input:** the unlabelled interval graph, each node carrying its in-degree and out-degree; edge labels (which transposition) are withheld.
- **Model:** a message-passing graph neural network, four rounds, predicting each coefficient of q, q^2, q^3, q^4 as a separate classification.
- **Result:** the coefficients were predicted with about 98% accuracy (Williamson, 2023) — evidence that $P_{x,y}$ is a function of the graph alone.

Geordie Williamson — representation theorist, University of Sydney; found the hypercube decomposition.

Saliency and the hypercube decomposition

- Gradient saliency ranks each edge by its effect on the prediction. The extremal reflections — those moving the first or last position — stand out far beyond chance.
- Williamson turned that into a decomposition of every interval, along its extremal reflections, into
  - a hypercube, from one set of extremal edges;
  - a subgraph isomorphic to an interval in S_{N-1}.
- From it $P_{x,y}$ is computed directly from the graph. The formula was checked on all intervals up to S_9 (over a million) and implies combinatorial invariance for symmetric groups (Blundell et al., 2022).



Interval $[(0,2,1,3),(2,3,0,1)]$ in S_4: hypercube piece (blue), interval-in-S_{N-1} piece (red).

Littlewood–Richardson and Kronecker coefficients

Two more multiplicities, both central in algebraic combinatorics:

- Littlewood–Richardson $c^{\lambda}_{\mu\nu}$ — V_λ inside $V_\mu \otimes V_\nu$ for GL_n; the Schur structure constants.
- Kronecker $g(\lambda, \mu, \nu)$ — S^λ inside $S^\mu \otimes S^\nu$ for the symmetric group; no combinatorial rule is known (Murnaghan's problem, open since 1938).

The value is hard to compute — #P-complete for Littlewood–Richardson, #P-hard for Kronecker, where even deciding which coefficients are nonzero is NP-hard. But the *support* — which are zero — is a classification problem, and a computer algebra system supplies exact labels in any quantity. Classifiers predict it well and sharpen as n grows, so more of the governing rule is still out there to find (the ML4AlgComb datasets, Kvinge et al. 2025).

The same pipeline in the Langlands program

Lanard and Mínguez (2024) used this pipeline to find an algorithm for Aubert–Zelevinsky duality on the p-adic groups SO_{2n+1} and Sp_{2n}, extending the Mœglin–Waldspurger algorithm from GL_n.

- Predicting the whole answer failed, so they predicted one piece at a time — the smallest segment — and read the formula off that.
- They calibrated the saliency on GL_n, where the algorithm is known, before trusting it on the classical groups, where it was not.

Conjecture generators

Graffiti: conjecturing inequalities between invariants

Graffiti (Fajtlowicz, mid-1980s) proposes inequalities among graph invariants.

- Keep a database of graphs; compute ~ 60 invariants (independence number, domination number, average distance, matching number, eigenvalues).
- Propose an inequality whenever it holds for every graph in the database.
- The Dalmatian heuristic keeps a new conjecture only if it is sharper on some graph than all kept so far.

Circulated as "Written on the Wall"; about eighty papers followed, by Chung, Erdős, Lovász, Seymour and others.

The Ramanujan Machine: continued fractions for constants

Raayoni et al. (*Nature* 590, 2021) generate conjectured *polynomial continued fractions* (PCFs) for fundamental constants — π, e, Catalan's constant G, and zeta values $\zeta(2) = \pi^2/6$, $\zeta(3)$:

$$x = a_0 + \cfrac{b_1}{a_1 + \cfrac{b_2}{a_2 + \cfrac{b_3}{a_3 + \ddots}}}, \qquad a_n, b_n \in \mathbb{Z}[n]$$

— the partial numerators b_n and denominators a_n are polynomials in n with integer coefficients.

- **Input:** a target constant, and bounded families of integer polynomials a_n, b_n.
- **Output:** an identity "constant = PCF", matched numerically — a conjecture, with no proof attached.

The Ramanujan Machine: how it searches

Both methods match numerical values only — no proof, no assumed structure.

MITM-RF (Meet-in-the-Middle):

- Input: the target constant; a library of candidate PCFs.
- Evaluate both sides to low precision; hash one side, look up the other.
- Re-check every hit at high precision to discard false matches.

Descent&Repel (gradient descent): minimise the gap between a PCF's value and the target over the integer coefficients; a "repel" step pushes away from solutions already found, to surface new ones.

Matches are tested to up to 2000 digits: a chance match in a 10^9 search at 50-digit accuracy has probability below 10^{-40}.

The Ramanujan Machine: $\zeta(3)$ and Catalan's constant

Extending the search produced previously-unknown continued fractions for $\zeta(3)$ (Apéry's constant) and Catalan's constant $G = 0.91596\ldots$:

$$\frac{12}{7\zeta(3)} = 2 - \cfrac{16 \cdot 1^6}{36 - \cfrac{16 \cdot 2^6}{160 - \cfrac{16 \cdot 3^6}{434 - \ddots}}} \qquad \frac{2}{2G-1} = 3 - \cfrac{6 \cdot 1^3}{13 - \cfrac{8 \cdot 2^3}{29 - \cfrac{10 \cdot 3^3}{51 - \ddots}}}$$

Both were matched from the numerical value alone, with no assumed structure, and were unproven when published; the Catalan-constant conjectures are still open.

The Ramanujan Machine: example formulas and status

Two formulas the machine found (both later proved):

$$\frac{e}{e-2} = 4 - \cfrac{1}{5 - \cfrac{2}{6 - \cfrac{3}{7 - \ddots}}}, \qquad \frac{4}{3\pi - 8} = 3 - \cfrac{1\cdot 1}{6 - \cfrac{2\cdot 3}{9 - \cfrac{3\cdot 5}{12 - \ddots}}}.$$

- The program finds the identity; it cannot prove it.
- Yamamoto (2024) proved 38 of the conjectures: each PCF reduces to a second-order linear difference equation, solved via a differential equation or Petkovšek's algorithm; 31 were generalised to wider families.
- Many others, including Catalan's-constant conjectures, remain open.

Murmurations of elliptic curves

Elliptic curves over $\mathbb{Q}$

- An **elliptic curve** over the rationals is the solution set of

$$E:\ y^2 = x^3 + ax + b, \qquad a, b \in \mathbb{Q},$$

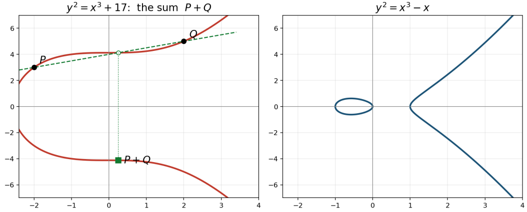
  together with one point O "at infinity".
- **Smoothness:** require the discriminant $\Delta = -16(4a^3 + 27b^2) \neq 0$, so the curve has no cusp or self-crossing.
- $E(\mathbb{Q})$ is the set of rational points (x, y) on E, plus O. The basic questions: how many are there, and how are they related?
- Over $\mathbb{R}$ the curve is one branch, or one branch plus a closed oval — $y^2 = x^3 - x$ has both.

The group law: chord and tangent

$E(\mathbb{Q})$ is an abelian group. A line meets the cubic in three points (with multiplicity); the operation $P + Q$ is built from that:

- **Sum.** The line through P, Q meets E in a third point R; reflect R across the x-axis to get $P + Q$.
- **Double.** For $2P$, use the *tangent* at P in place of the chord, then reflect.
- **Identity.** O, the point at infinity where all vertical lines meet, is neutral: $P + O = P$.
- **Inverse.** $-P$ is the reflection of P; the line through them is vertical, so its third point is O and $P + (-P) = O$.

Commutative and associative; $nP = P + \cdots + P$ builds infinitely many points from a few.



Left: the sum $P + Q$ on $y^2 = x^3 + 17$ via the chord and the reflection. Right: the two-component locus $y^2 = x^3 - x$.

Mordell's theorem and the rank

- **Mordell (1922):** $E(\mathbb{Q})$ is a finitely generated abelian group,
$$E(\mathbb{Q}) \;\cong\; \mathbb{Z}^r \;\oplus\; E(\mathbb{Q})_{\mathrm{tors}},$$
with $E(\mathbb{Q})_{\mathrm{tors}}$ finite.
- The **rank** $r \geq 0$ is the number of independent infinite-order points that generate, by the group law, all but finitely many rational points.
- Rank 0: finitely many rational points; rank ≥ 1: infinitely many.
- Computing the rank exactly is hard — no algorithm is proven to terminate for every curve.
- But it can be *predicted* from the a_p with high accuracy — statistics, not proof. This is the quantity the rest of the story turns on.

Louis Mordell (1888–1972): proved $E(\mathbb{Q})$ finitely generated, 1922. Sadleirian Chair, Cambridge.

From $E(\mathbb{Q})$ to $E(\mathbb{F}_p)$: why count mod p

- **The rank is hard to reach directly.** It lives over $\mathbb{Q}$, the rational points can be infinite, and no algorithm is proven to compute it for every curve.
- **Reduce mod p.** For a good prime $p \nmid \Delta$, reduce the coefficients a, b mod p: the result is an elliptic curve over the finite field $\mathbb{F}_p = \{0, \ldots, p-1\}$. Now x, y each take only p values, so the points, plus O, form a *finite* group $E(\mathbb{F}_p)$ —a directly computable count.
- **The link.** Reducing coordinates is a group homomorphism $E(\mathbb{Q}) \to E(\mathbb{F}_p)$ (good p): each rational point maps to a mod-p point. So $E(\mathbb{Q})$ is reflected in the finite groups, and the rank shows up as a *bias* in the counts N_p —more rational points, more points mod p on average. BSD (next) makes this link exact.

a_p and the L-function

- For each good prime, the point count gives

$$N_p = \#E(\mathbb{F}_p), \qquad a_p = p + 1 - N_p,$$

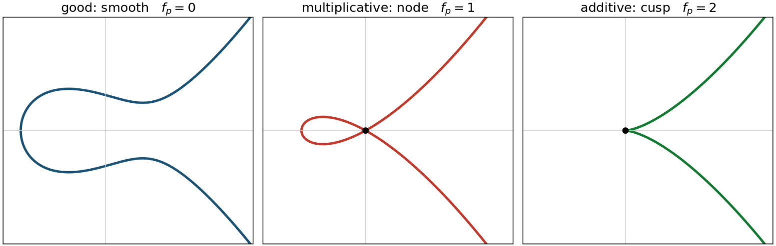
  the **Frobenius trace** — how far N_p strays from the expected $p+1$. Hasse: $|a_p| \leq 2\sqrt{p}$.
- The (a_p) assemble into the L-**function**

$$L(E,s) = \prod_{p\,\text{good}} \left(1 - a_p p^{-s} + p^{1-2s}\right)^{-1} = \sum_{n\geq 1} a_n n^{-s}, \quad \mathrm{Re}(s) > \tfrac{3}{2}.$$

- By the modularity theorem (Wiles; Taylor–Wiles; Breuil–Conrad–Diamond–Taylor) $L(E,s)$ extends to all of $\mathbb{C}$. Its center $s=1$ is where the arithmetic concentrates.

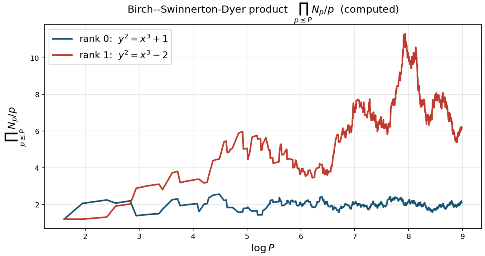
Reduction types and the conductor

- At a bad prime $p \mid \Delta$ the reduced curve is singular: a **node** (two tangent lines) is *multiplicative* reduction, a **cusp** (one) is *additive*; good reduction is smooth.
- The **conductor** is $N(E) = \prod_p p^{f_p}$ with $f_p = 0$ (good), 1 (mult.), 2 (additive, $p \geq 5$).
- At $p = 2, 3$ a wild term can raise it ($f_2 \leq 8$, $f_3 \leq 5$); in general $f_p = v_p(\Delta_{\min}) + 1 - m_p$ ($m_p = $ #components of the special fibre; Ogg, via Tate's algorithm).
- N square-free $\iff$ E semistable. The conductor orders curves by size and sets the murmuration bands.



The Birch–Swinnerton-Dyer conjecture, found by computer

- Birch and Swinnerton-Dyer, on the **EDSAC-2** at Cambridge (1958–62), computed N_p and studied $\prod_{p \le P} \frac{N_p}{p}$, finding it grows like $C(\log P)^r$ — higher rank, more points mod p.
- **Conjecture.** $\operatorname{rank} E(\mathbb{Q}) = \operatorname{ord}_{s=1} L(E,s)$: algebraic rank $=$ analytic rank.
- A Clay Millennium Prize Problem; proved only for analytic rank ≤ 1 (Gross–Zagier, Kolyvagin), open in general.
- So BSD itself was **found by computer** — the conjecture read off the data, not derived from theory.



$\prod_{p \le P} N_p/p$ vs $\log P$ (computed): rank-1 grows, rank-0 stays bounded.

Birch–Swinnerton-Dyer: the precise statement

For $E/\mathbb{Q}$ with $r = \operatorname{ord}_{s=1} L(E,s)$, the conjecture has two parts:

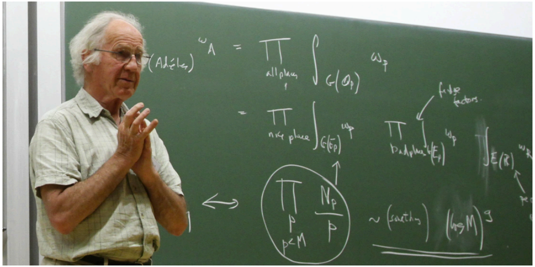
- **Rank:** $r = \operatorname{rank} E(\mathbb{Q})$.
- **Leading coefficient:** $\displaystyle\lim_{s\to 1} \frac{L(E,s)}{(s-1)^r} = \frac{\Omega_E \cdot \operatorname{Reg}(E) \cdot \#Ш(E) \cdot \prod_p c_p}{\left(\#E(\mathbb{Q})_{\mathrm{tors}}\right)^2}$.

Invariants (minimal model):

- Ω_E — real period $\int_{E(\mathbb{R})} |\omega|$ (ω the Néron differential).
- $\operatorname{Reg}(E)$ — regulator: $\det$ of the Néron–Tate height pairing on a basis of $E(\mathbb{Q})/\mathrm{tors}$ ($= 1$ if $r = 0$).
- $\#Ш(E)$ — order of the Tate–Shafarevich group (failure of the local–global principle; conjectured finite).
- c_p — Tamagawa number $[E(\mathbb{Q}_p) : E^0(\mathbb{Q}_p)]$ ($= 1$ at good p); $\prod_p c_p$ finite.
- $\#E(\mathbb{Q})_{\mathrm{tors}}$ — order of the torsion subgroup.

Birch and Swinnerton-Dyer

Working from their EDSAC-2 point counts at Cambridge in the early 1960s, Birch and Swinnerton-Dyer read off the rank–L-function correlation that became the conjecture.



Bryan Birch (b. 1931), British; University of Oxford.
Also known for Birch's theorem.

Sir Peter Swinnerton-Dyer (1927–2018), British;
University of Cambridge; 16th Baronet.

The machine: EDSAC 2

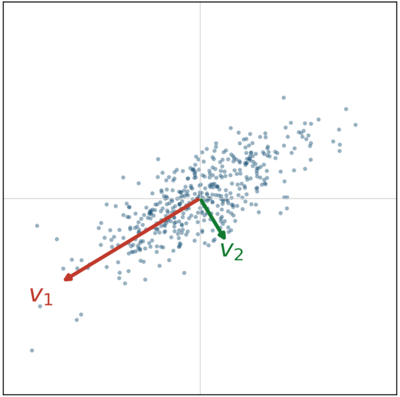
- A vacuum-tube (valve) computer at the Cambridge Mathematical Laboratory, running from **1958** — successor to EDSAC (1949).
- Built under Maurice Wilkes with David Wheeler: the **first computer with a microprogrammed control unit**, and the first bit-slice architecture.
- Birch and Swinnerton-Dyer ran their elliptic-curve point counts on it; the conjecture came out of those runs.
- Its small store and slow speed limited the runs to small-conductor curves.

EDSAC 2 in 1960, Cambridge Mathematical Laboratory. Wikimedia Commons.

Principal component analysis (PCA)

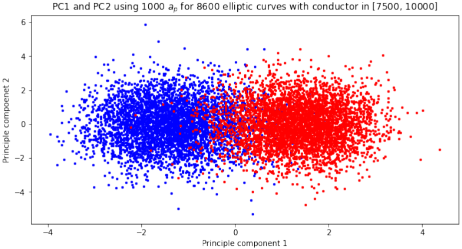
- An **unsupervised** linear dimensionality-reduction method: orthogonal directions (principal components) ordered by the variance they capture.
- Centre the points; the eigenvectors $v_1, v_2, \ldots$ of the covariance $C = \frac{1}{n} X^\top X$ (eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots$) are the components — v_1 the most-variance direction, $v_2 \perp v_1$ next, λ_i the variance along v_i (the SVD of X).
- Project onto the top two, $x \mapsto (v_1 \cdot x,\ v_2 \cdot x)$, to plot high-dimensional data in 2-D.
- Unsupervised: uses the data, not the labels, so any clusters are intrinsic.

Principal components of a point cloud



A machine-learning rank classifier

- The **LMFDB** (L-functions and Modular Forms Database) lists elliptic curves with rank, conductor, and (a_p) — labelled data.
- He, Lee, and Oliver, with Lee's undergraduate Pozdnyakov (2020–22), trained classifiers to predict rank from $(a_{p_1}, \ldots, a_{p_{1000}})$.
- Ranks separate cleanly: logistic regression reaches precision 0.96 on $\{0,1\}$ and 0.9998 on $\{1,2\}$; principal component analysis (PCA), a top-variance projection, separates them visually.
- So the a_p carry the rank. *What* in them? Pozdnyakov averaged them and plotted.



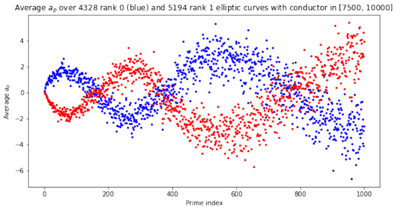
PCA of the a_p vectors, conductor $[7500, 10000]$; ranks separate. He–Lee–Oliver–Pozdnyakov, arXiv:2204.10140.

The murmuration: the averaged statement

- The **conductor** $N(E)$ is built from the bad primes (dividing Δ); it orders curves by size.
- In a band $[N_1, N_2]$, let $\mathcal{E}_r$ be the rank-r curves. Average the n-th Frobenius trace:

$$f_r(n) = \frac{1}{\#\mathcal{E}_r} \sum_{E \in \mathcal{E}_r} a_{p_n}(E).$$

- **Finding (2022):** f_0, f_1 trace two mirrored oscillating curves, amplitude and period growing with n; approximately scale-invariant across bands.
- Named *murmurations* for the look of starling flocks. Empirical: no formula, no proof.



f_0, f_1, conductor $[7500, 10000]$ (arXiv:2204.10140).

A starling murmuration. Geograph, CC BY-SA.

The murmuration discoverers

- **Yang-Hui He** (London Institute for Mathematical Sciences) — brought machine learning to these number-theory datasets.
- **Kyu-Hwan Lee** (University of Connecticut) and **Thomas Oliver** (University of Westminster) — set up the rank-classification experiments on L-function data.
- **Alexey Pozdnyakov** (then Lee's undergraduate student) — averaged the a_p by rank and plotted them: the murmuration.

Published in *Experimental Mathematics* 34 (2025) (arXiv:2204.10140).

Yang-Hui He, London Institute for Mathematical Sciences. Wikimedia Commons.

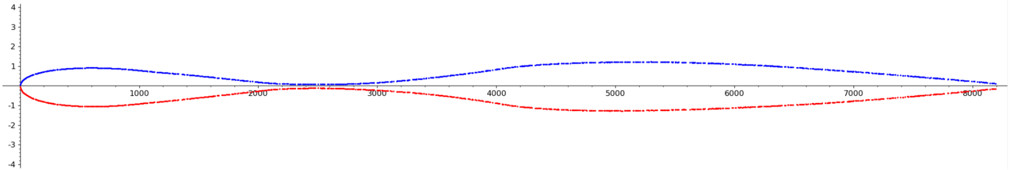
From elliptic curves to modular forms

- Sutherland (MIT) reproduced murmurations on $> 10^9$ curves and in far more general L-functions — the phenomenon is not special to elliptic curves.
- The clean setting is **modular forms**: a holomorphic function on the upper half-plane transforming under $\Gamma_0(N)$ (level N, weight k); a *newform* is a Hecke eigenform genuinely of level N, with Fourier coefficients $a_f(p)$.
- By modularity an elliptic curve *is* a weight-2 newform with the same a_p — so the elliptic-curve murmuration is the weight-2 case.
- Each f has a **root number** $\varepsilon(f) = \pm 1$ (sign of its functional equation), the analogue of rank parity — group forms by $\varepsilon(f)$. Normalized coefficient $\lambda_f(p) = a_f(p)/p^{(k-1)/2}$, $|\lambda_f(p)| \leq 2$ (the analogue of $a_p/\sqrt{p}$).

Sutherland's limit curve

The elliptic-curve murmuration averaged a_p over rank classes and plotted it against the prime index — and was scale-invariant. Sutherland makes that exact:

- Fix a band of levels $N \in [N, cN]$ and one prime P that scales with the level, $P \sim N$.
- **Root-number-weighted average:** multiply each $a_f(P)$ by its sign $\varepsilon(f)$, then average over all weight-k newforms in the band — the signs make the bias add up instead of cancel.
- As $N \to \infty$ this converges to one **continuous curve** depending only on $y = P/N$ (prime size $\div$ level size).



Average of $\sqrt{P}\,\lambda_f(P)\,\varepsilon(f)$ over levels $[2^{13}, 2^{14}]$ vs $y = P/N$. Courtesy of Sutherland, in arXiv:2310.07681.

Zubrilina's theorem: the ingredients

- $H^{\mathrm{new}}(N,k)$: the weight-k, trivial-character **newforms** of level N — the genuinely-level-N Hecke eigenforms (a finite set).
- $a_f(p)$: the p-th Fourier coefficient; $\lambda_f(p) = a_f(p)/p^{(k-1)/2}$ its normalization ($|\lambda_f(p)| \le 2$); $\varepsilon(f) = \pm 1$ the root number.
- The averaged quantity is a **root-number-weighted mean**: sum $\sqrt{P}\,\lambda_f(P)\,\varepsilon(f)$ over all square-free levels $N \in [X, X+Y]$ and all $f \in H^{\mathrm{new}}(N,k)$, divided by the count of those f.
- $y = P/X$: the prime-to-level ratio (the scale variable). Square-free N is technical — it makes the trace formula simplify.

Zubrilina's theorem (2023): the density formula

With those ingredients, as $X \to \infty$:

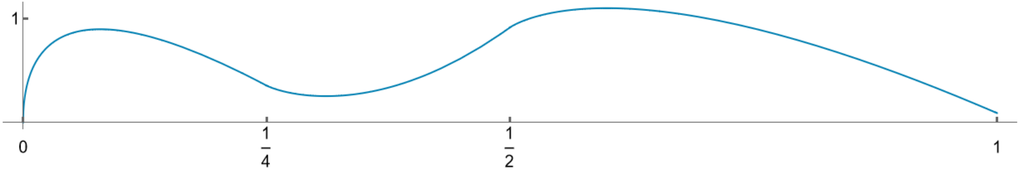
$$\frac{\sum_N \sum_f \sqrt{P}\,\lambda_f(P)\,\varepsilon(f)}{\sum_N \sum_f 1} \;=\; \mathcal{M}_k(y) \;+\; \text{(power-saving error)},$$

$$\mathcal{M}_k(y) = \frac{\alpha(-1)^{k/2-1}}{k-1} \sum_{1\le r\le 2\sqrt{y}} \nu(r)\sqrt{4y-r^2}\,U_{k-2}\Big(\tfrac{r}{2\sqrt{y}}\Big) + \frac{\beta}{k-1}\sqrt{y} - \gamma\,\delta_{k=2}\,y.$$

U_{k-2} = Chebyshev polynomial of the second kind ($U_n(\cos\theta) = \sin((n{+}1)\theta)/\sin\theta$); $\alpha, \beta, \gamma, \nu(r)$ = explicit products over primes (Euler products); $\delta_{k=2} = 1$ iff $k = 2$. Sarnak: the *Zubrilina murmuration density formula* (arXiv:2310.07681). The elliptic-curve murmuration is $k = 2$.

The density $\mathcal{M}_k$: proof idea and shape

- **Proof idea.** Apply the **Eichler–Selberg trace formula** — which writes a sum of Hecke eigenvalues over a space of modular forms in terms of *class numbers* of binary quadratic forms — to a composition of Hecke and Atkin–Lehner operators. The average becomes an average of class numbers, evaluated in short intervals by the class number formula.
- **Shape.** $\mathcal{M}_k$ is continuous and oscillating, with kinks at $y = n^2/4$; it grows like $\sqrt{y}$ near 0 and like $y^{1/4}$ as $y \to \infty$, with $\mathcal{M}_k(y)/y^{1/4}$ asymptotically almost periodic in $\sqrt{y}$ (a convergent sum of Bessel-J terms).
- Integrating $\mathcal{M}_k$ recovers Sutherland's dyadic-band averages; for $k = 2$ the limit is an explicit piecewise formula on $[0, 1]$.



The piecewise weight-2 limit. Zubrilina, arXiv:2310.07681.

The murmuration is now a family of theorems

Zubrilina's weight-k density was the first proof. The phenomenon has since been established for further families:

- **Dirichlet characters** — Lee, Oliver, Pozdnyakov (arXiv:2307.00256).
- **Weight aspect** — Bober, Booker, Lee, Lowry-Duda; level fixed, weight $\to \infty$ (arXiv:2310.07746).
- **Maass forms** — the non-holomorphic case (arXiv:2409.00765).
- **Ordered by height** — the same curves ordered by naive height (arXiv:2504.12295).
- **Function fields** — the function-field analogue (arXiv:2603.13802).

Computing the rank vs predicting it

- **Certified** rank: descent (e.g. `mwrank`, Sage) — correct when it finishes, but it can stall when the Tate–Shafarevich group is large; no algorithm is proven to terminate for every curve.
- **Predicting** the rank from the a_p is easier — and the a_p determine it, through the explicit formula

$$-\frac{L_E'(s)}{L_E(s)} = \sum_{n\geq 1} \frac{c_n \Lambda(n)}{n^s} = \frac{r}{s-1} + \cdots,$$

  where Λ is the von Mangoldt function and $c_p = a_p$. The analytic rank r is the order of the pole at $s = 1$; by BSD it equals the algebraic rank.
- So a weighted prime sum of the a_p estimates r — prediction, not proof.

Mestre–Nagao sums

Explicit prime sums that estimate the analytic rank (Mestre 1982, Nagao 1992; the list S_0–S_6 collected in arXiv:2207.06699). The basic one:

$$S_0(B) = \frac{1}{\log B} \sum_{\substack{p<B \\ \text{good}}} \frac{a_p(E)\,\log p}{p}.$$

- **Kim–Murty:** under the Riemann Hypothesis for L_E, if the limit exists, $\lim_{B\to\infty} S_0(B) = -r + \frac{1}{2}$.
- Variants: Nagao's S_3 (logarithmic derivative of the partial Euler product at $s=1$, large for large rank); Bober's S_6 (from the explicit formula, $\to r$ under RH, but infeasible beyond a small range).
- A sum returns a *number*; reading a rank off it needs a decision rule and a conductor-dependent cutoff.

Learning the rank: trees and a CNN

- **Gradient-boosted trees** (Alessandretti–Baronchelli–He, 2019, arXiv:1911.02008): 2.5M Cremona curves; an ensemble of decision trees finds correlations among rank, Weierstrass coefficients, conductor, regulator, Tamagawa number, and the order of the Tate–Shafarevich group.
- **A 1-D CNN** (Kazalicki–Vlah, 2022, arXiv:2207.06699) learns the rank from a curve's a_p sequence.
- On the LMFDB it beats every Mestre–Nagao sum: Matthews correlation coefficient (a balanced score, max 1) 0.9958 — only 0.25% misclassified — vs 0.87 for the best single sum. A net fed all sums at once beats any one sum.
- On a harder set (conductor up to 10^{30}, rank to 10) accuracy drops; very large conductors need more a_p.

The CNN rank classifier: input, output, product

- **Input.** One curve becomes a $3 \times \pi(N)$ matrix; each column is a prime $p < N$ $(N = 10^3, 10^4, 10^5)$:
  - row 1: $a_p/\sqrt{p} \in [-2, 2]$ (Hasse) — this row carries the rank;
  - row 2: the normalized conductor $\log N_E / \log N_{\max}$;
  - row 3: the prime's position in the sequence.
- **Output.** A probability for each rank class $0, 1, 2, \ldots$; the prediction is the largest. A classification, not a single number.
- **Training.** Supervised on LMFDB curves whose rank is already known (certified by descent); minimise the cross-entropy loss by back-propagation. The 1-D convolution slides one learned filter along the prime axis.
- **Product.** A fixed-weight function $f_\theta\colon (a_p$ matrix$) \mapsto$ rank — a predictor with no certificate. It estimates the analytic rank ($=$ algebraic under BSD); certifying the rank still needs descent.

The CNN design (Kazalicki–Vlah)

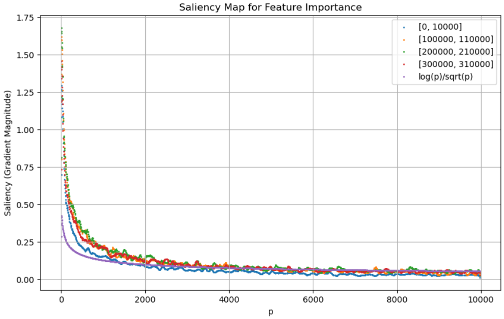
Design rationale: a Mestre–Nagao sum applies the same rule to every a_p — a shared weighting — and a 1-D convolution is exactly such a shared filter slid along the primes. The network is a stack of 1-D convolutions:

- **Preparatory layer:** widens the input from 3 channels to 64.
- **Input layers** (L_1): keep 64 channels and the length.
- **Reducing layers** (L_2): stride 2, each halves the length; L_2 is set so the length reaches 1.
- **Output layers** (L_3): keep 64 channels, length 1.
- **Final dense layer:** maps $64 \to$ the number of rank classes.

Each convolution is followed by ReLU and batch normalization; one kernel size KS throughout. The net is fixed by (L_1, L_2, L_3, KS) — the LMFDB $p < 10^4$ model used $(0, 11, 3, 17)$, found by Bayesian optimization over 500 variants.

The network's saliency: murmuration early, Mestre–Nagao late

- Saliency w_p — the gradient of the output with respect to each a_p — shows what the CNN relies on.
- **Early** in training the rank-0-vs-rank-1 saliency shows the *murmuration* oscillation.
- **Later** the rank-0 saliency flattens and weight moves to small primes, matching the Mestre–Nagao weighting $\log p/\sqrt{p}$.
- Murmurations matter more for small conductors. Test accuracy 99.85% on $[0, 10^4]$, 98.57% on $[3\times10^5, +10^4]$.



Trained CNN saliency across four conductor ranges vs the Mestre–Nagao weighting $\log p/\sqrt{p}$ (purple). arXiv:2603.17681.